基于同态散列验证的跨竖井联邦学习实用多密钥安全聚合

Practical Multi-Key Secure Aggregation With Homomorphic Hash Verification for Cross-Silo Federated Learning

Chenxu Fan, Yuwen Chen, *Member, IEEE*, Qin Jiang, Haiyang Yu, *Member, IEEE*, and Zhen Yang, *Member, IEEE*

***Abstract*—Federated learning (FL) is a collaborative machine learning framework that enables multiple clients to jointly train a global model without exposing their local data. However, securely combining model updates from different devices while maintaining efficient communication is challenging. Existing data aggregation methods incorporate homomorphic encryption and threshold secret sharing to protect data privacy, which often require a lot of communication costs. To solve this, we propose a new multi-private key secure aggregation method, which simplifies the key exchange process. We further introduce a clipping-aware packed parameter encoding mechanism, reducing computation by approximately 25% and communication by 55% compared to unencoded transmission. Extensive experiments show our overall framework outperforms existing baselines with nearly 20% faster aggregation and 45% lower communication overhead, while bounding the IID accuracy degradation within 4%. These gains make secure federated learning highly viable for cross-silo deployments.**

***Index Terms*—Federated learning (FL), deep learning, data privacy, distributed learning, data reconstruction attack.**

I. Introduction

FEDERATED learning (FL) [1], [2] is a collaborative machine learning framework that enables clients to build a global model collaboratively while keeping their raw data on local devices. This is particularly suitable for privacy-sensitive fields like healthcare and finance, where data confidentiality is of utmost importance. Instead of exchanging raw data, federated learning transmits only model updates, helps prevent data breaches and complies with privacy regulations. It also reduces communication costs by sending only model parameters rather than large datasets. However, despite these advantages, federated learning is vulnerable to security threats, one of the most concerning being gradient inversion attacks [3], [4], [5], [6], in which adversaries attempt to infer sensitive user private data from the shared model updates, which can compromise user privacy.

To ensure data privacy in federated learning, researchers have proposed various secure aggregation schemes [7], [8], [9], [10], [11]. These methods prevent external attackers from performing gradient inversion attacks, where adversaries aim to infer private training samples from the shared gradients. In addition, secure aggregation ensures that only the aggregated model update is visible to the central server, preventing access to any individual client's data, thereby enhancing privacy. However, many secure aggregation schemes rely on homomorphic encryption [9], [11] or threshold secret sharing [7], [8], which introduce significant communication overhead. Homomorphic encryption allows aggregating encrypted data without decrypting it, providing strong security but requiring extensive computational resources and increased communication between clients and the server. Likewise, threshold secret sharing enhances confidentiality by splitting sensitive data among multiple clients, but it also leads to substantial communication costs. These limitations introduce considerable communication overhead and scalability challenges, making the deployment of federated learning in large-scale environments more difficult. Therefore, designing communication-efficient and privacy-preserving aggregation mechanisms remains an important research problem.

Secure aggregation schemes employ homomorphic encryption [9], [10], [12], [13] to encrypt local gradients, ensuring only legitimate parties can access the aggregated results. In several such schemes, clients are required to generate public-private key pairs, typically established through a Diffie-Hellman key exchange protocol facilitated by a trusted third party. However, this mechanism introduces significant computational and communication overhead, as each client must securely negotiate and distribute encryption keys before contributing their model updates. The high communication costs make these aggregation schemes less practical for large-scale federated learning settings, particularly when network bandwidth is limited or when devices

Received 17 June 2025; revised 19 March 2026; accepted 26 March 2026. Date of publication 31 March 2026; date of current version 2 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302020 and Grant 62302229, in part by the R&D Program of Beijing Municipal Education Commission under Grant KM202210005028, and in part by the National Key R&D Program of China under Grant 2022YFB3103100. Recommended for acceptance by X. Yuan. *(Corresponding author: Yuwen Chen.)*

Chenxu Fan, Yuwen Chen, Haiyang Yu, and Zhen Yang are with the Data Mining & Security Lab and Beijing Key Laboratory of Trusted Computing, College of Computer Science, Beijing University of Technology, Beijing 100021, China (e-mail: 1907890307@emails.bjut.edu.cn; yuwen.chen@bjut.edu.cn; yuhaiyang@bjut.edu.cn; yangzhen@bjut.edu.cn).

Qin Jiang is with the Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing University of Information Science and Technology, Nanjing 211544, China (e-mail: qinj@nuist.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TMC.2026.3679423, provided by the authors.

Digital Object Identifier 10.1109/TMC.2026.3679423

1536-1233 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

have constrained computational resources. Furthermore, these schemes assume that no collusion occurs among clients or between clients and the aggregation server. If collusion does take place, the security guarantees of homomorphic encryption-based schemes may be compromised, posing additional risks to data privacy.

Secure aggregation schemes often rely on threshold secret sharing to preserve the confidentiality of individual client updates before aggregation [7], [8], [11]. Some of these schemes enable a double-masking mechanism, wherein local gradients are obscured using random masks that are designed to cancel out during the aggregation process. However, the use of secret sharing introduces significant communication overhead because each client must distribute shares of its private key to all other clients. The amount of data exchanged will grow rapidly when the number of participants increases, making the scheme less scalable. Additionally, in each training iteration, clients are required to regenerate masking secrets and perform interactive secret-sharing protocols, further increasing computational and communication costs. These frequent interactions introduce delays and make real-time federated learning applications more challenging. Moreover, a core trade-off exists between enhanced privacy and system efficiency: increasing the number of secret shares enhances privacy protection, but it also imposes greater communication and computational burdens on the system, making deployment in large-scale scenarios more difficult.

In addition to the communication overhead introduced by secure aggregation mechanisms, federated learning itself suffers from substantial communication and computational burdens when training large deep learning models [14], [15]. As the model size increases, the gradients exchanged during each training iteration can become massive, sometimes reaching gigabyte-scale. With the rise of federated training of Large Language Models (LLMs) [16], [17], this problem becomes particularly acute. Because LLMs usually has tens of billions of parameters, updating all these parameters requires considerable bandwidth and computational resources, which can be a bottleneck, thereby limiting scalability and training efficiency. To tackle this challenge, we introduce a clipping-aware packed parameter encoding method and an *Accuracy-Adaptive Model Transmission* technique that enhances communication efficiency while maintaining model privacy. Our method not only reduces communication costs but also strengthens resistance to gradient inversion attacks, providing better protection against privacy breaches. Our contributions mainly reflect in the following.

1) *Practical Multi-Private Key Secure Aggregation:* We design a secure aggregation protocol for federated learning in which each client employs an independent private key for encrypted model updates, enabling privacy-preserving aggregation without revealing individual client gradients.

2) *Verifiable Aggregation via Linear Homomorphic Hash:* We integrate a linear homomorphic hash mechanism that allows clients to verify the correctness of the aggregated result, ensuring that any manipulation by the aggregation server can be detected.

3) *Clipping-Aware Packed Parameter Encoding:* We propose a clipping-aware parameter packing and encoding method that reduces the communication cost of model updates while preserving model accuracy.

4) *Security Evaluation Against Attacks:* We analyze the proposed encoding mechanism under gradient inversion and data poisoning attacks, demonstrating improved resistance to these threats while maintaining model utility.

The organization of this paper is as follows: Section II discusses related works. Section III covers the preliminaries. In Section IV, we present the multi-private key aggregation scheme, and Section V provides a discussion on security. Section VI compares core features of different schemes, followed by the evaluation section in Section VII. Section VIII is the conclusion.

II. Related Work

Shokri and Shmatikov proposed a pioneering federated learning framework in which multiple participants collaboratively train a deep learning model without exposing their raw data [1]. The FedAvg algorithm [2] enables high-quality model training using relatively few communication rounds. Federated learning is often considered privacy-preserving since local data remains on the device and is never transmitted. However, federated learning's privacy can still be compromised by gradient inversion attacks [3], [4]. These attacks exploit the sharing of local model updates (gradients) between participants, allowing attackers to reconstruct the original training sample, thus undermining the privacy of FL.

A variety of data aggregation schemes [7], [8], [9], [10] have been introduced to improve privacy protection in federated learning. Some of these methods use homomorphic encryption [9], [18], [10], [19], [20], [21] to encrypt the local model updates (gradients). Homomorphic encryption enables direct computation over ciphertexts, eliminating the need for decryption during processing and thereby preserving data confidentiality. However, as deep learning models grow larger, encrypting gradients becomes computationally expensive and leads to high communication costs, making it less efficient. Other methods, like threshold secret sharing [7], [8], [10], require clients to share parts of their encryption keys with other clients before aggregation. While this also helps protect privacy, it requires significant communication since each client must share keys with almost all other clients. Some other approaches go a step further by decentralizing the trust placed in the central server, distributing it between two servers that operate within separate trust domains [22], [23], [24]. Both of these techniques result in high communication costs, which are a challenge for large-scale federated learning.

A significant challenge in federated learning lies in the communication overhead associated with transmitting gradients. Lin et al. [25] discovered that 99.9% of the gradients exchanged in distributed stochastic gradient descent are redundant, meaning they do not contribute much. This highlights the need for more efficient communication methods in federated learning. Shokri and Vitaly [1] observed that certain model parameters have a larger impact on the neural network's loss function and thus receive more substantial updates in each training round. Using

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYMBOLS IN THIS STUDY

Symbol	Description
$\mathcal{S}$	The parameter server.
B_u	The BCP ciphertext of user u.
m	A generic plaintext message.
$\mathbb{G}$	The multiplicative cyclic group of prime order.
n	Total number of users.
b	Bit width of the quantized integer.
λ	The quantization scaling factor.
$(a_1, a_2, \ldots, a_t)$	A sequence of packing coefficients.
d	The dimension of the local model.
t	Number of quantized gradients packed into a single ciphertext.
k	Number of packed blocks (or ciphertexts) per user.
$(g_1, g_2, \ldots, g_d)$	Generators of $\mathbb{G}$ used for LHH verification.
(x_u, r_u)	Random keys for encrypting the local model.
(pk_{x_u}, pk_{r_u})	The public keys employed for encryption.
dk_{γ_u}	The BCP decryption key of user u.
dk_{γ}	The aggregated BCP decryption key.
sk_h	Secret key used to generate the LHH generators.
(d_u^{sk}, d_u^{pk})	Signing and verification key pair of user u.
σ_u	The digital signature generated by user u.
h_u	The LHH verification tag of user u.
$\boldsymbol{\omega}$	The aggregated global model.
$\boldsymbol{\omega}_u$	The local model trained by user u.
$\boldsymbol{\omega}'_u$	The fully connected layers of the local model.
ω_{ui}	The i-th component of user u's local model.
Δ_u	The clipped model update of user u.
δ	The variation in local model accuracy between consecutive rounds.
τ	The predefined threshold for triggering AAMT.
$\boldsymbol{\mu}_u$	The quantized model update of user u.
$\boldsymbol{y}_u$	The packed representation of user u's quantized model.
$\boldsymbol{\gamma}_u$	The ciphertext vector of the packed model $\boldsymbol{y}_u$.

selective parameters of all the gradients can still maintain acceptable model performance in federated learning.

The verifiability of aggregation is also important for maintaining data privacy in federated learning [26]. On one hand, the central server may have a financial incentive to perform faster computations, even if they are not entirely accurate, in order to save computational resources [27]. On the other hand, studies by Hitaj et al. [28] and Fowl et al. [29] show that the central server could manipulate the global model to launch more powerful gradient inversion attacks, allowing the server to directly access user data from the gradient updates without solving complex inverse problems. To mitigate this risk, methods such as VerifyNet [26], VOSA [27], and VPPFL [30] have been developed to verify computations over joint inputs from multiple users, while maintaining privacy.

To enhance the efficiency of verifiable federated learning, Cai et al. proposed ESVFL [31], a verifiable and privacy-preserving FL scheme that introduces dual cloud servers for cross-verification without interaction between users. This method reduces computation and communication overhead by eliminating user interaction in both encryption and verification phases. It also efficiently handles user dropouts without extra cost and ensures security even when collusion exists between servers. Similarly, Li et al. proposed OVP-FL [32], a framework that supports outsourced privacy-preserving and verifiable FL. The scheme ensures that users can detect incorrect aggregation results returned by servers with negligible overhead, rendering it well-suited for devices with limited resources and dropout scenarios.Wu et al. introduced ESAFL [33], a homomorphic encryption-based framework tailored for cross-silo FL. ESAFL leverages an additive homomorphic encryption algorithm to achieve one-step decryption. This significantly reduces communication and computation costs while preserving model accuracy. In another study, Wang et al. designed a lightweight secure aggregation protocol using symmetric homomorphic encryption and a homomorphic signature mechanism to support both privacy protection and verifiability [34]. Their solution divides verification into offline and online phases, further reducing computational overhead and making it well-suited for practical applications with client dropout.

Existing schemes address either communication efficiency or aggregation verifiability, but not both simultaneously. Packing-based methods such as MSA [35] improve efficiency via parameter packing, yet do not consider clipping-aware encoding or its impact on privacy against gradient inversion attacks. Verifiable aggregation schemes, including VerifyNet [26], VOSA [27], and VPPFL [30], ensure correctness but rely on conventional encoding, leading to higher communication overhead. In contrast, our scheme integrates clipping-aware packed encoding with lightweight homomorphic hash verification, achieving both communication efficiency and verifiability. A detailed comparison is given in Table III.

III. PRELIMINARIES

Table I introduces symbols utilized in this scheme in detail. In this section, we discussed the threat model, the BCP cryptosystem, the gradient inversion attack, the data poisoning attack, the linear homomorphic hash and the proposed clipping-aware packed parameter encoding.

A. Threat Model

We consider an adversary that may control the central server $\mathcal{S}$ and up to $n-2$ users, capable of arbitrarily deviating from the protocol without breaking the underlying cryptographic primitives. Our system comprises three entity types:

Server and Users (Collusion): The untrusted server $\mathcal{S}$ may attempt to inspect messages, disrupt aggregation, or launch inference attacks (e.g., gradient inversion [29], [36]). The adversary can collude with up to $n-2$ malicious users to infer private updates. Consequently, we require at least two users to remain honest per round; otherwise, an $n-1$ coalition could trivially recover the remaining user's masked update.

Trusted Authority (TA): The TA is strictly an offline entity responsible only for generating system-wide public parameters and authentication keys during setup. It does not participate in training and has no access to model updates, per-round randomness, or masking secrets.

Justification and Limitations: The TA assumption is practical and widely adopted in *cross-silo* federated learning, where participating entities are stable organizations (e.g., hospitals or research institutes) that typically rely on consortium-managed initialization infrastructure. However, we acknowledge the reliance on a centralized TA as a system limitation. For fully decentralized deployments, the TA's setup functionality could

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

alternatively be implemented via distributed key generation (DKG) among the participants.

B. Multi-Key Extension of the BCP Cryptosystem

The BCP cryptosystem [37] is originally designed for a single public–private key pair. To support secure aggregation among multiple users, we extend BCP to a *multi-key* setting, where each user holds an independent secret key and no single party possesses all private keys. This extension preserves the additive homomorphism of BCP and enables distributed decryption through key aggregation. Below, we describe the construction in a way that directly corresponds to the multi-key encryption and decryption requirements.

1) *Multi-Key Generation:* A trusted authority generates system parameters following the standard BCP setup [37]. Specifically, two large safe primes $p = 2p_1 + 1$ and $q = 2q_1 + 1$ are selected, and $N = pq$ is computed. A generator $g \in \mathbb{Z}^*_{N^2}$ is derived as $g = \alpha^2 \bmod N^2$ for random α.

Each user u independently samples a secret key $x_u \in \mathbb{Z}_{Np_1q_1}$ and publishes the corresponding public key $pk_{x_u} = g^{x_u} \bmod N^2$. The aggregated public key is defined as

$$pk_x = \prod_{u=1}^{n} pk_{x_u} = g^{\sum_{u=1}^{n} x_u} \bmod N^2. \tag{1}$$

This aggregated key enables encryption under multiple independent secret keys without revealing any individual private information.

2) *Multi-Key Encryption:* Given a plaintext $m_u \in \mathbb{Z}_N$, user u selects a fresh random value r_u and encrypts m under the aggregated public key pk_x as

$$B_u = (pk_x)^{r_u}(1 + m_u N) \bmod N^2. \tag{2}$$

This encryption procedure is identical to standard BCP encryption, except that the public key is formed by aggregating the contributions of all users.

3) *Homomorphic Aggregation and Distributed Decryption:* Due to the additive homomorphism of BCP, ciphertexts from different users can be aggregated by multiplication:

$$\prod_{u=1}^{n} B_u = g^{(\sum_{u=1}^{n} x_u)(\sum_{u=1}^{n} r_u)} \left(1 + \sum_{u=1}^{n} m_u N\right) \bmod N^2. \tag{3}$$

For decryption, no single user holds the full secret key. Instead, each user computes a partial decryption share

$$dk_{\gamma_u} = \left(g^{\sum_{u=1}^{n} r_u}\right)^{x_u} \bmod N^2. \tag{4}$$

The final decryption key is obtained by aggregating all shares:

$$dk_\gamma = \prod_{u=1}^{n} dk_{\gamma_u} = g^{(\sum_{u=1}^{n} x_u)(\sum_{u=1}^{n} r_u)} \bmod N^2. \tag{5}$$

4) *Correctness of Multi-Key Decryption:*

Theorem 1: The proposed multi-key extension of the BCP cryptosystem correctly recovers the aggregated plaintext $\sum_{u=1}^{n} m_u$ through distributed decryption, while preserving the original correctness guarantees of BCP.

We show that the multi-key construction does not alter the decryption structure of the original BCP scheme and that the aggregated plaintext can be correctly recovered.

Each user u encrypts m_u as

$$B_u = (pk_x)^{r_u}(1 + m_u N) \bmod N^2,$$

where $pk_x = g^{\sum_{u=1}^{n} x_u}$. By the additive homomorphism of BCP, multiplying all ciphertexts yields

$$\prod_{u=1}^{n} B_u = g^{(\sum_{u=1}^{n} x_u)(\sum_{u=1}^{n} r_u)} \left(1 + \sum_{u=1}^{n} m_u N\right) \bmod N^2.$$

For decryption, each user computes a decryption share $dk_{\gamma_u} = (g^{\sum_u r_u})^{x_u}$. Aggregating all shares gives

$$dk_\gamma = \prod_{u=1}^{n} dk_{\gamma_u} = g^{(\sum_u x_u)(\sum_u r_u)} \bmod N^2.$$

Dividing the aggregated ciphertext by the aggregated decryption key cancels the random masking term:

$$\frac{\prod_{u=1}^{n} B_u}{dk_\gamma} = 1 + \sum_{u=1}^{n} m_u N \bmod N^2.$$

Applying the standard BCP decoding function $L(x) = (x - 1)/N$ correctly recovers $\sum_{u=1}^{n} m_u$.

Importantly, the above decryption process is algebraically identical to standard BCP decryption under a single secret key $x = \sum_{u=1}^{n} x_u$. Therefore, the correctness of the proposed multi-key decryption directly follows from the correctness of the original BCP cryptosystem [37], and no additional assumptions are introduced.

5) *BCP Encryption and Decryption Interfaces:* For notational convenience, and to simplify the presentation of the protocol flow in Fig. 1, we define the encryption and decryption interfaces of the multi-key BCP cryptosystem as follows.

Definition 1 (Multi-Key BCP Encryption): Given a plaintext $m_u \in \mathbb{Z}_N$, an aggregated public key pk_x, and a fresh randomness r_u generated by user u, the BCP encryption function is defined as

$$\mathbf{BCP.enc}(m_u, pk_x, r_u) \to B_u, \tag{6}$$

where $B_u = (pk_x)^{r_u}(1 + m_u N) \bmod N^2$.

Definition 2 (Multi-Key BCP Decryption): Given an aggregated ciphertext $\boldsymbol{\gamma} = \prod_{u=1}^{n} B_u$ and the aggregated decryption key $dk_\gamma = \prod_{u=1}^{n} (pk_r)^{x_u}$, the BCP decryption function outputs the aggregated plaintext as

$$\mathbf{BCP.dec}(\boldsymbol{\gamma}, dk_\gamma) \to m, \tag{7}$$

where the plaintext is recovered by $L(\boldsymbol{\gamma}/dk_\gamma \bmod N^2) = ((\boldsymbol{\gamma}/dk_\gamma - 1)/N)$.

C. Gradient Inversion Attacks in Federated Learning

Gradient inversion attacks [3], [4], [5], [6] are a way for malicious actors to exploit the shared gradients and potentially reconstruct sensitive data from them, even though the raw data is never explicitly transmitted. In a gradient inversion attack, the attacker attempts to "invert" these gradients to recover sensitive

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

Practical multi-key secure aggregation with homomorphic hash verification for cross-silo federated learning

- **Setup**

  TA:

  – Choose big integers N, p, q, p_1, q_1, g for BCP cryptosystem. Then, Select $\mathbb{G}$ and generator $g' \in \mathbb{G}$.
  – Select a pair of signature keys (d_u^{sk}, d_u^{pk}) for each user u. Generate a key sk_h for LHH.
  – Publish N, g, g', d_u^{pk}, the number of users n and the bit width of the quantized integer b. Send d_u^{sk} and sk_h to corresponding users.

- **Round 0 Advertise Keys**

  User u:

  – Randomly select x_u, r_u and use them to generate $pk_{x_u} = g^{x_u}$, $pk_{r_u} = g^{r_u}$.
  – Send pk_{x_u}, pk_{r_u} to TA.

  TA:

  – Receive messages from users. Aggregate them as $pk_x = \prod_{u=1}^{n} pk_{x_u} = g^{\sum_{u=1}^{n} x_u}$ and $pk_r = \prod_{u=1}^{n} pk_{r_u} = g^{\sum_{u=1}^{n} r_u}$.
  – Broadcast pk_x, pk_r to all users.

- **Round 1 Masked Inputs**

  User u:

  – Receive the aggregated public keys pk_x, pk_r from TA.
  – Train model with local dataset and get the local model $\boldsymbol{\omega}_u$.
  – *Accuracy-Adaptive Model Transmission*: verify whether the accuracy of the $\boldsymbol{\omega}_u$ on the validation set satisfies the requirements. If unsatisfied, use $\boldsymbol{\omega}_u$ in the following step. Otherwise, using the $\boldsymbol{\omega}_u{}'$.
  – Generate a sequence $g_1, g_2 \dots g_d$ through $g_i = H(sk_h \| i), i = 1, 2, \dots, d$.
  – Clip the selected update to obtain Δ_u
  – Generate the quantized gradients as: $\boldsymbol{\mu}_u \leftarrow \mathbf{Qua}(\lambda, \Delta_u, b)$, where λ is a coefficient generated by user.
  – Generate the verification information as: $h_u \leftarrow \mathbf{LHH.Hash}(\boldsymbol{\mu}_u)$.
  – Generate the signature of the verification information h_u by $\sigma_u \leftarrow \mathbf{SIG.sign}(h_u, d_u^{sk})$.
  – Encode the quantized gradients using *Clipping-Aware Packed Parameter Encoding*: $\boldsymbol{y}_u \leftarrow \mathbf{Encode}(\boldsymbol{\mu}_u)$.
  – Encrypt the quantized gradients $\boldsymbol{y}_u$ to get $\boldsymbol{\gamma}_u = \boldsymbol{B}_u \leftarrow \mathbf{BCP.enc}(\boldsymbol{y}_u, pk_x, r_u)$.
  – Send the $\{\boldsymbol{\gamma}_u, h_u, \sigma_u\}$ to server.

  Server:

  – Receive messages from users. Aggregate them and get $\boldsymbol{\gamma} = \prod_{u=1}^{n} \boldsymbol{\gamma}_u$.
  – Aggregates the signature to get $\sigma = \sum_{u=1}^{n} \sigma_u$.
  – Broadcast $\{\boldsymbol{\gamma}, \sigma\}$ and the set of hash h_u to all users.

- **Round 2 Aggregate Keys**

  User u:

  – Receive the aggregation results and signatures from server. Get $dk_{\gamma_u} = (pk_r)^{x_u} = g^{x_u \sum r_u}$.
  – Send the dk_{γ_u} to server.

  Server:

  – Receive messages from users. Aggregate them and get $dk_\gamma = \prod_{u=1}^{n} dk_{\gamma_u}$.
  – Broadcast dk_γ to all users.

- **Round 3 Unmask and Verify**

  User u:

  – Receive the dk_γ from server.
  – Check whether $\mathbf{SIG.bver}(\sigma, h_1, h_2, \dots, h_n, d_1^{pk}, d_2^{pk}, \dots, d_n^{pk}) \stackrel{?}{=} 1$, if failed, abort the aggregation.
  – Decrypt the $\boldsymbol{\gamma}$ by $\boldsymbol{y} \leftarrow \mathbf{BCP.dec}(\boldsymbol{\gamma}, dk_\gamma)$.
  – Decode $\boldsymbol{y}$ as: $\boldsymbol{\mu} \leftarrow \mathbf{Decode}(\boldsymbol{y})$.
  – Dequantize $\boldsymbol{\mu}$ as: $\Delta' \leftarrow \mathbf{DeQua}(\lambda, \boldsymbol{\mu}, b)$ to obtain the aggregated model update $\boldsymbol{\omega}$.
  – Check whether the $\mathbf{LHH.Hash}(\boldsymbol{\mu}) \stackrel{?}{=} \mathbf{LHH.Eval}(h_1, \dots, h_n, \underbrace{1, \dots, 1}_{\text{n items}})$. If the result is true, accept the $\boldsymbol{\omega}$. Otherwise, abort it.

Fig. 1. The details of the proposed practical multi-key secure aggregation with homomorphic hash verification for cross-silo federated learning.

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ILLUSTRATION OF CLIPPING STRATEGIES

Clipping Strategy	Clipping Function	Clipping Norm
FClip	$\Delta = \Delta / \max(1, \frac{\|\|\Delta\|\|_2}{C})$	C
LClip	$\Delta = \Delta_j / \max(1, \frac{\|\|\Delta\|\|_2}{C})$	$C = \sqrt{\sum_{j=1}^{\mathcal{L}} C_j^2}$

information used to create them, such as private user data. The attacker does this by analyzing the patterns in the gradients and using mathematical techniques to approximate the original data that produced them.

D. Poisoning Attack in Federated Learning

A data poisoning attack [38], [39], [40] in federated learning occurs when clients unknowingly train on compromised local data, causing the global model to learn malicious patterns or degrade in performance. Such attacks are difficult to detect because the server only observes gradients rather than raw data, and the resulting updates may appear statistically benign while still introducing harmful effects after aggregation.

Gradient clipping is a standard technique in federated learning, primarily used to stabilize training by preventing exploding gradients. Table II outlines commonly adopted clipping methods. The approach works by limiting the norm of gradients or model updates to a predefined threshold, thereby reducing the influence of outliers. Although not originally designed to defend against poisoned data, gradient clipping can offer partial mitigation by restricting the impact of anomalous updates—particularly when the poisoned data induces large or skewed gradients during training [41], [42].

E. LHH Verification

We adopt the Linear Homomorphic Hash (LHH) [43] to verify the correctness of aggregation. LHH enables efficient correctness checking by ensuring that the hash of an aggregated vector equals the homomorphic combination of individual hashes. This property is particularly suitable for secure aggregation, as it avoids revealing individual updates while supporting public verification. An LHH scheme consists of three algorithms: key generation, hashing, and evaluation.

- $\text{LHH.HGen}(1^\kappa, 1^d)$ takes a security parameter κ and a vector dimension d as input, and outputs public parameters $\mathbb{G}$ and $\{g_i\}_{i=1}^d$. Here, $\mathbb{G}$ is a cyclic group of prime order p', and $\{g_i\}_{i=1}^d$ are independent generators in $\mathbb{G}$.
- For user u, Given a d-dimensional vector $x^{(u)} = (x_{u1}, x_{u2}, \ldots, x_{ud})$, the hashing algorithm computes

$$\text{LHH.Hash}(x^{(u)}) = h_u = \prod_{i=1}^{d} g_i^{x_{ui}}. \tag{8}$$

  The resulting hash compactly commits to all coordinates of x while preserving linearity.
- $\text{LHH.Eval}(h_1, h_2 \ldots h_n, \alpha_1, \alpha_2 \ldots \alpha_n)$ is the Evaluation function. Input n linear homomorphic hash value h_i and d coefficients α_i, this algorithm will output the combination of them as: $\text{LHH.Eval}(h_1, \ldots, h_n, \underbrace{\alpha_1, \alpha_2 \ldots \alpha_n}_{\text{n items}}) = h_* = \prod_{i=1}^{d} {h_i}^{\alpha_i}$. In particular, when all coefficients satisfy $\alpha_i = 1$, clients can verify the correctness of an aggregated model $\sum_{i=1}^{n} x^{(i)}$ by checking whether:

$$\text{LHH.hash}(\sum_{i=1}^{n} x^{(i)}) \stackrel{?}{=} \text{LHH.Eval}(h_1, \ldots, h_n, \underbrace{1, \ldots, 1}_{\text{n items}})$$

F. Digital Signature

Let $\mathbb{G}$ be a cyclic multiplicative group of prime order p', g' is a generator of $\mathbb{G}$. A bilinear pairing is defined as a map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$. Suppose there exists a hash function H that maps bit arbitrary-length bit strings from $\{0,1\}^*$ into elements of $\mathbb{G}$:

$$H : \{0,1\}^* \to \mathbb{G}$$

The BLS signature scheme [44] consists of following algorithms:

- $\textbf{SIG.gen}(1^\kappa)$ outputs a key pair (d^{sk}, d^{pk}), where $d^{sk} \in \mathbb{Z}_{p'}^*$ and $d^{pk} = (g')^{d^{sk}}$.
- $\textbf{SIG.sign}(m, d^{sk})$. Given a message m, the signature is computed as $\sigma = d^{sk} \cdot H(m)$.
- $\textbf{SIG.ver}(\sigma, m, d^{pk})$. A signature σ on message m is valid if

$$e(\sigma, g') \stackrel{?}{=} e(H(m), d^{pk}) \tag{9}$$

- $\textbf{SIG.bver}(\sigma_{agg}, m_1, m_2, \ldots, m_u, d_1^{pk}, d_2^{pk}, \ldots, d_u^{pk})$. Given signatures $\{\sigma_i\}_{i=1}^u$ on messages $\{m_i\}_{i=1}^u$, let $\sigma_{\text{agg}} = \sum_{i=1}^u \sigma_i$. The verifier checks

$$e(\sigma_{\text{agg}}, g') \stackrel{?}{=} \prod_{i=1}^{u} e(H(m_i), d_i^{pk}). \tag{10}$$

IV. THE PROPOSED SCHEME

We first introduce a clipping-aware packed encoding module, which serves as a privacy- and efficiency-enhancing front-end for the secure aggregation protocol by jointly handling gradient clipping, quantization, and coefficient-based packing. Building upon this encoding design, we further present an Accuracy-Adaptive Model Transmission (AAMT) mechanism that dynamically regulates the amount of transmitted information according to training accuracy, thereby reducing communication overhead without compromising aggregation correctness. Together, these two components form the core design enhancements of our scheme and are seamlessly integrated into the subsequent secure aggregation protocol.

A. Clipping-Aware Packed Parameter Encoding

To enhance both computational efficiency and privacy protection during the aggregation phase of federated learning, we propose a clipping-aware packed parameter encoding method, which consists of three steps: clipping, quantization and packed encoding. The computational/communication efficiency is improved. And as shown in our experiments, the clipping-aware

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Select Encoding Coefficients.

Input: n (users), t (plaintext capacity per ciphertext), b (bits), N^2 (scheme bits)
Output: Coefficients $\boldsymbol{a} = (a_1, a_2, \ldots, a_t)$

$M \leftarrow n(2^b - 1)$; // Max sum per data type
$a_1 \leftarrow 1$; // Smallest positive integer
for $i = 2$ *to* t **do**
  $S \leftarrow \sum_{j=1}^{i-1} a_j$;
  $a_i \leftarrow S \cdot M + 1$; // Ensure $a_i > \sum_{j=1}^{i-1} a_j M$
Verify $\sum_{i=1}^{t} a_i M < N$; // Within bounds
return $\boldsymbol{a}$;

Algorithm 2: Encode Local Data.

Input: Local data $\boldsymbol{\mu}_u = (\mu_{u1}, \ldots, \mu_{ud})$, packing coefficients $\boldsymbol{a} = (a_1, \ldots, a_t)$
Output: Packed values $\boldsymbol{y}_u = (y_{u1}, \ldots, y_{uk})$

$k \leftarrow \lceil d/t \rceil$;
Initialize $y_{u1}, \ldots, y_{uk} \leftarrow 0$;
for $\ell \leftarrow 1$ **to** d **do**
  $j \leftarrow \lceil \ell/t \rceil$;
  $i \leftarrow (\ell - 1) \bmod t + 1$;
  $y_{uj} \leftarrow y_{uj} + a_i \cdot \mu_{u\ell}$;
return *Packed values* $\boldsymbol{y}_u = (y_{u1}, \ldots, y_{uk})$

Algorithm 3: Decode Aggregated Values.

Input: Packing coefficients $\boldsymbol{a} = \{a_1, \ldots, a_t\}$, aggregated values $\boldsymbol{y} = (y_1, \ldots, y_k)$
Output: Aggregated sums $sum_1, \ldots, sum_d$

for $\ell \leftarrow d$ **to** 1 **do**
  $j \leftarrow \lceil \ell/t \rceil$;
  $i \leftarrow (\ell - 1) \bmod t + 1$;
  $tem \leftarrow y_j \bmod a_i$;
  $sum_\ell \leftarrow (y_j - tem)/a_i$;
  $y_j \leftarrow tem$;
return $sum_1, \ldots, sum_d$

encoding increases resistance to gradient inversion attacks and poison attacks.

Clipping Process: First, the encoding process begins with gradient clipping, which serves to normalize the scale of model updates and limit the influence of outliers. We adopt two common clipping strategies, as summarized in Table II: flat clipping (FClip) and layer-wise clipping (LClip) [45]. In FClip, all model layers are clipped equally, while each layer is treated as a separate vector and clipped independently in LClip. We simply set $C_j = \sqrt{C_{\mathcal{L}}}$ for all layers j, where $\mathcal{L}$ is the number of layers of the model [45].

Quantization Process: Second, the clipped gradients are quantized. The quantization scheme used is similar to [46]. Because encryption schemes typically support only integer operations, each clipped floating-point value Δ_{ui} is first scaled using a quantization factor λ and converted into an integer. Then, the result is shifted by adding 2^{b-1} to handle negative values. The quantization operation for the i-th dimension of user u's update is shown in (11). Let $\mu_{ui} \leq 2^b - 1$, where b represents the bit width of the quantized integer. The corresponding dequantization process is given by (12):

$$\mu_{ui} \leftarrow \mathbf{Qua}(\lambda, \Delta_{ui}, b) \tag{11}$$

$$\Delta'_{ui} \leftarrow \mathbf{DeQua}(\lambda, \mu_{ui}, b) \tag{12}$$

Regarding the quantization parameter λ, its effect on model accuracy is marginal. Since λ only scales the floating-point number before rounding, its influence is restricted to a few least significant decimal places. On the other hand, the bit width b defines the precision used to store each value after quantization. Since b does not change the underlying scale of the gradient and is strictly standardized across all users in our study, its impact on model accuracy is also negligible.

Packed Encoding Process: We introduce a parameter packing scheme based on remainder theory. The key idea is to encode multiple quantized gradients into a single ciphertext, so that secure aggregation can be performed with a single homomorphic addition. This approach significantly reduces communication overhead and improves aggregation efficiency under the homomorphic encryption framework. To enable joint decoding after aggregation, all users use the same method to generate a common sequence of packing coefficients $\boldsymbol{a} = (a_1, a_2, \ldots, a_t)$, which ensures that each encoded segment occupies a unique position within the packed value and can therefore be aggregated without interference.

Assume each quantized gradient is represented with a bit-length of b. For a user u with d dimensions of local data $\boldsymbol{\mu}_u = (\mu_{u1}, \mu_{u2}, \ldots, \mu_{ud})$, the aggregated result for the i-th dimension over n users is $\sum_{u=1}^{n} \mu_{ui}$. Suppose t dimensions of data $(\mu_{u1}, \mu_{u2}, \ldots, \mu_{ut})$ can be packed into one ciphertext. To safely pack these values, we must consider the largest possible aggregated magnitude. The maximum value for each packed component is bounded by (13).

$$M = n(2^b - 1) \geq \sum_{u=1}^{n} \mu_{ui} \tag{13}$$

To prevent overlaps among different packed segments, the packing coefficients must satisfy the non-overlapping constraint in (14). This ensures that even after summing encrypted packed values from all users, each segment remains uniquely recoverable.

$$\sum_{j=1}^{i-1} a_j M < a_i \tag{14}$$

Additionally, the packed plaintext remains within the BCP plaintext space $\mathbb{Z}_N$, while the ciphertext naturally resides in $\mathbb{Z}_{N^2}$, leading to the global constraint in (15). This prevents plaintext overflow and guarantees correct homomorphic computation.

$$\sum_{i=1}^{t} a_i M < N \tag{15}$$

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

To satisfy both constraints while minimizing growth in coefficients, we construct $\boldsymbol{a} = (a_1, a_2, \ldots, a_t)$ using the recurrence relation in (16). The detailed selection procedure is given in Algorithm 1.

$$a_1 = 1, \qquad a_i = \sum_{j=1}^{i-1} a_j M + 1 \tag{16}$$

By combining (13), (15), and (16), we obtain the upper bound in (17), which further yields the maximum number of gradients that can be packed into a single ciphertext, shown in (18).

$$\sum_{i=1}^{t} a_i M = (M+1)^t - 1 < N \tag{17}$$

$$t \leq \left\lfloor \frac{\log(N+1)}{\log\big(1 + n(2^b - 1)\big)} \right\rfloor \tag{18}$$

For a user u with d categories of local data $\boldsymbol{\mu}_u = (\mu_{u1}, \mu_{u2}, \ldots, \mu_{ud})$, he can encode his data into $k = \lceil d/t \rceil$ integers for encryption. The resulting packed value $\boldsymbol{y}_u = (y_{u1}, \ldots, y_{UK})$ is a k length array described by Algorithm 2.

Finally, the end-to-end encoding and decoding operations are summarized in (19) and (20), respectively. The d represents the size of the local model. The decoding process performs repeated modular reductions and divisions based on the selected coefficients and does not require decryption keys or additional interaction.

$$\mathbf{Encode}(\mu_{u1}, \mu_{u2}, \ldots, \mu_{ud}) \rightarrow \boldsymbol{y}_u \tag{19}$$

$$\mathbf{Decode}(\boldsymbol{y}) \rightarrow \mu_1, \mu_2, \ldots, \mu_d \tag{20}$$

Theorem 2 (Decoding Correctness): Let the packing coefficients $\boldsymbol{a} = (a_1, \ldots, a_t)$ be constructed according to Algorithm 1, and let $\boldsymbol{y} = (y_1, \ldots, y_k)$ denote the aggregated packed values obtained by summing the ciphertexts of all users. Assume that each local entry satisfies $0 \leq \mu_{u\ell} \leq 2^b - 1$ and let $sum_\ell = \sum_{u=1}^{n} \mu_{u\ell}$. Then Algorithm 3 correctly recovers all aggregated values $\{sum_1, \ldots, sum_d\}$.

We first characterize the structure of the aggregated packed values. For each block index $j \in \{1, \ldots, k\}$, the aggregated plaintext can be written as

$$y_j = \sum_{i=1}^{t} a_i \cdot sum_{(j-1)t+i}, \tag{21}$$

where undefined indices with $(j-1)t + i > d$ are implicitly zero. By (13), each aggregated component satisfies $0 \leq sum_\ell \leq M$.

Block-level Decodability: From the construction in (16), the coefficients satisfy

$$a_i > sum_{r=1}^{i-1} a_r M, \tag{22}$$

which guarantees that within each block y_j, the contribution of $a_i \cdot sum_{(j-1)t+i}$ dominates all lower-order terms. Therefore, for a fixed block j, applying the operation $y_j \bmod a_i$ removes the i-th and all higher-order components, isolating the lower-order part.

Sequential Decoding: Algorithm 3 iterates over $\ell = d, d-1, \ldots, 1$. For each ℓ, it determines the corresponding block index $j = \lceil \ell/t \rceil$ and the intra-block position $i = (\ell - 1) \bmod t + 1$. Let $y_j^{(\ell)}$ denote the current value of y_j at this step. By (22), we have

$$sum_\ell = \frac{y_j^{(\ell)} - (y_j^{(\ell)} \bmod a_i)}{a_i} \tag{23}$$

and updating $y_j^{(\ell-1)} \leftarrow y_j^{(\ell)} \bmod a_i$ removes the decoded component without affecting the remaining lower-order ones.

Global Correctness: Since different blocks y_j are independent and each block is decoded correctly from higher to lower indices, the entire sequence $\{sum_1, \ldots, sum_d\}$ is recovered without interference. All operations are purely arithmetic (modulus and division), and no overflow occurs due to the global constraint in (15).

Hence, Algorithm 3 correctly decodes all aggregated values.

B. Accuracy-Adaptive Model Transmission

In federated learning, the global model is typically divided into several blocks based on the sequential structure of network layers, where each block comprises a set of adjacent layers in the architecture [47]. Under this setting, users may transmit only a subset of model parameters to the central server while still achieving comparable training performance. Based on it, we proposed the *Accuracy-Adaptive Model Transmission (AAMT)* method. A user u will test the accuracy of $\boldsymbol{\omega}_u$ on his local validation dataset to determine whether it meets the requirements. If the accuracy is satisfactory, user u uploads only the parameters associated with the fully connected layers of $\boldsymbol{\omega}_u$, denoted as $\boldsymbol{\omega}'_u$. This is the because the fully connected layers are primarily used for classification tasks. And the convolutional layers are primarily used for feature extraction. Otherwise, user u must upload the entire local model $\boldsymbol{\omega}_u$. For convenience, $\boldsymbol{\omega}_u = (\omega_{u1}, \omega_{u2}, \ldots, \omega_{ud})$ represents a d-dimensional vector, where $\boldsymbol{\omega}'_u$ is a vector of smaller length than $\boldsymbol{\omega}_u$. Through it, our scheme can greatly reduce the communication costs caused by large ciphertexts in homomorphic encryption.

AAMT Triggering and Security Discussion: AAMT is triggered locally by each user based on their model's accuracy variation (δ) on a validation set between consecutive rounds t and $t-1$. If $|\delta| < \tau$ (empirical thresholds are detailed in Table VIII), only the fully connected (FC) layers are transmitted; otherwise, the full model update is uploaded.

Security-wise, this adaptive payload selection merely reveals a coarse local convergence state, introducing no additional leakage of raw data. Regarding manipulation, while malicious clients can deliberately alter their local transmission strategy (e.g., forcing a full upload), this deviation strictly increases their own communication overhead without compromising the protocol's security or other clients' privacy.

C. Secure Aggregation Protocol

Our scheme includes four phases: Setup phase, Advertise Keys phase, Masked Inputs phase, Aggregate Keys phase, and

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

Unmasking and Verify phase. The brief flow detailed scheme is shown in Fig. 1.

1) Setup: We consider a setting with n users involved in the scheme, denoted as $\{U|u_i(i=1,2,\ldots,n)\}$. During the setup phase, a Trusted Authority (TA) selects two large prime numbers, p and q. The TA then calculates $N=pq$. Additionally, the TA ensures that both p and q satisfy the safe prime condition by finding p_1 and q_1 such that $p=2p_1+1$ and $q=2q_1+1$.

Next, the TA selects a random element $g \in \mathbb{Z}^*_{N^2}$, and a multiplicative cyclic group $\mathbb{G}$ of prime order p'. Let g' be the generator of $\mathbb{G}$.

Then, TA generates a pair of signature keys (d_u^{sk}, d_u^{pk}) for each user, where $d_u^{pk}=(g')^{d_u^{sk}}$. Besides, TA generates the key sk_h to calculate the sequence used in LHH.

Finally, the TA publishes N, g, g', d_u^{pk}, the total number of users n and the bit-length of gradients b to both the users and the server, while sk_h is only distributed to users and d_u^{sk} is sent to the corresponding user.

2) Advertise Keys: In this phase, the user u first randomly selects x_u, r_u from $\mathbb{Z}_{Np_1q_1}$. Subsequently, the user computes the following public keys: $pk_{x_u}=g^{x_u}$ and $pk_{r_u}=g^{r_u}$. Then, u sends pk_{x_u}, pk_{r_u} to the TA.

Upon receiving the messages from all users, the TA aggregates all pk_{x_u}, pk_{r_u} as: $pk_x=\prod pk_{x_u}=g^{\sum x_u} \mod N^2$ and $pk_r=\prod pk_{r_u}=g^{\sum r_u}$. Finally, the TA broadcasts all the aggregated keys pk_x, pk_r to all users.

3) Masked Inputs: In this phase, each user u receives the global model $\boldsymbol{\omega}$ from the server, using their private local dataset to obtain a local model $\boldsymbol{\omega}_u$. To improve efficiency, our scheme utilizes the Accuracy-Adaptive Model Transmission (AAMT) to determine whether this round should send the full set of model parameters $\boldsymbol{\omega}_u$ or only the selected subset $\boldsymbol{\omega}'_u$.

Then each user u uses sk_h to generate a sequence of elements $g_1, g_2, \ldots, g_d$ in $\mathbb{G}$. With the shared key sk_h generated by TA. The sequence is computed as

$$g_i = H(sk_h \parallel i), \qquad i=1,2,\ldots,d.$$

Before encryption, the gradients are processed using Clipping-Aware Packed Parameter Encoding. To enable encryption under the BCP cryptosystem, each local model $\boldsymbol{\omega}_u$ is first quantized. Given a quantization coefficient λ and a gradient bit-length b, the user obtains an integer-valued vector $\boldsymbol{\mu}_u=(\mu_{u1},\mu_{u2},\ldots,\mu_{ud})$ according to (11), which converts real-valued updates into a form suitable for cryptographic processing.

Since the bit-length b of each quantized model is much smaller than the plaintext capacity of BCP, we further apply a packing strategy to improve efficiency. We pack t consecutive entries into one integer with a sequence $\boldsymbol{a}=(a_1,a_2,\ldots,a_t)$, which generated by all users through the same method. Thus $k=\lceil d/t \rceil$, which means d types of data will be encode into k integers. We can obtain $\boldsymbol{y}_u=(y_{u1},\ldots,y_{UK})$, where

$$y_{uj}=\sum_{i=1}^{t} a_i \cdot \mu_{u,(j-1)t+i}, \qquad j=1,\ldots,k.$$

Each packed value satisfies $y_{uj} < N$, ensuring its validity as a plaintext under the BCP cryptosystem.

User u then encrypts the packed vector $\boldsymbol{y}_u$. For simplicity, we denote the transmitted ciphertext component by $\boldsymbol{\gamma}_u = B_u$. The ciphertext vector is given by $\boldsymbol{\gamma}_u=(\gamma_{u1},\gamma_{u2},\ldots,\gamma_{UK})$, where

$$\begin{aligned}\gamma_{ui} &= (pk_x)^{r_u}(1+y_{ui}N) \bmod N^2 \\ &= g^{r_u\sum x_u}(1+y_{ui}N) \bmod N^2, \qquad i=1,\ldots,k.\end{aligned}$$

To enable verification on the aggregated model, user u generates a linear homomorphic hash over the local model. Specifically, after quantization, user u computes

$$h_u = g_1^{\mu_{u1}} \cdot g_2^{\mu_{u2}} \cdots g_d^{\mu_{ud}},$$

Where $\boldsymbol{\mu}_u=(\mu_{u1},\ldots,\mu_{ud})$ denotes the quantized local model. To prevent server malicious tampering h_u, our scheme uses the BLS signature scheme. User u generates the signature $\sigma_u = d_u^{sk} \cdot H(h_u)$ and sends $\{\boldsymbol{\gamma}_u, h_u, \sigma_u\}$ to the server.

After collecting the ciphertexts and signatures from all users, the server performs ciphertext aggregation and acquires $\boldsymbol{\gamma}=(\gamma_1,\gamma_2,\ldots\gamma_k)$. Since $pk_x=g^{\sum_{u=1}^{n} x_u}$ is publicly broadcast by the TA, each user can locally compute $(pk_x)^{r_u}$ without learning any individual secret. The γ_i is computed by:

$$\begin{aligned}\gamma_i=\prod_{u=1}^{n}\gamma_{ui}&=\prod_{u=1}^{n}\left(g^{r_u\sum x_u}(1+y_{ui}N)\right) \mod N^2 \\ &= g^{\sum r_u \sum x_u}(1+N\sum_{u=1}^{n} y_{ui}) \mod N^2.\end{aligned}$$

Also, the server aggregates the signature and acquires $\sigma=\sum_{u=1}^{n}\sigma_u$ Finally, the server broadcasts the aggregated results $\boldsymbol{\gamma}$, the aggregated signature σ and the list of verification information $\{h_u\}$ to all users.

4) Aggregate Keys: To decrypt the aggregation results, user u needs to compute the decryption key dk_{γ_u}. dk_{γ_u} is defined as follows:

$$dk_{\gamma_u}=(pk_r)^{x_u}=g^{x_u\sum r_u} \mod N^2.$$

User u then sends dk_{γ_u} to the server. After collecting the decryption keys from all users, the server aggregates them as:

$$\begin{aligned}dk_\gamma=\prod_{u=1}^{n} dk_{\gamma_u}&=\prod_{u=1}^{n} g^{x_u\sum r_u} \mod N^2 \\ &= g^{\sum x_u \sum r_u} \mod N^2,\end{aligned}$$

In the final step, the server broadcasts the aggregated decryption key dk_γ to users for further recovery of the plaintext.

5) Unmask and Verify: In this phase, user u first checks the correctness of the h_u with the aggregated signature σ through $\mathbf{SIG.bver}(\sigma, h_1, h_2, \ldots, h_n, d_1^{pk}, d_2^{pk}, \ldots, d_n^{pk}) \stackrel{?}{=} 1$. If it fails, abort the aggregation result.

Then user u utilizes the aggregated decryption key received from the server to obtain the aggregated plaintext. The ciphertext

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF COMPUTATION COMPLEXITY AND SECURITY FEATURES

Protocols	PSA [7]	VerifyNet [26]	EDA [48]	FSA [46]	VOSA [27]	VPPFL [30]	MSA [35]	Ours
Clipping-Aware	×	×	×	✓	×	×	×	✓
Packed Parameter Encoding	-	-	×	✓	×	×	✓	✓
Accuracy-Adaptive Transmission	×	×	×	×	×	×	×	✓
Communication round	4	4	3	3	3	4	3	3
Aggregation verification	×	✓	×	×	✓	✓	×	✓

is decrypted as follows:

$$\begin{aligned}\boldsymbol{y} &= \frac{\left(\frac{\gamma}{dk_\gamma} - 1\right)}{N} \mod N^2 \\ &= \frac{\left(\frac{g^{\sum x_u \sum r_u (1+N\sum \boldsymbol{y}_u)}}{g^{\sum x_u \sum r_u}} - 1\right)}{N} \mod N^2 \\ &= \sum_{u=1}^{n} \boldsymbol{y}_u\end{aligned}$$

Subsequently, u decodes and dequantizes $\boldsymbol{y}$ to retrieve $\boldsymbol{\omega}$ using Algorithm 3. To ensure the integrity of the aggregation result, u verifies whether

$$\mathbf{LHH.Hash}(\boldsymbol{\mu}) \stackrel{?}{=} \mathbf{LHH.Eval}(h_1, \ldots, h_n, \underbrace{1, \ldots, 1}_{\text{n items}})$$

If the equality holds, u accepts the aggregation result. Otherwise, u aborts the aggregation result for this round. The process continues iteratively until the task is completed. LHH verification is performed in the quantized integer domain to ensure algebraic consistency with homomorphic aggregation.

Client Dropout and Partial Participation: Secure aggregation is performed over a fixed set of participants in each round. If a client drops out after advertising its aggregation key or during the unmasking and verification phase, the round is aborted and restarted with the remaining active users. This design targets cross-silo federated learning with stable participants, where mid-round dropouts are rare. Accordingly, the protocol assumes reliable participation within each round and does not guarantee correctness under arbitrary dropout patterns.

V. COMPARISON OF THE FEATURES

The feature comparison is summarized in Table III. Our scheme supports aggregation result verification, effectively preventing adversaries from tampering with the aggregated outcomes to compromise user's privacy. Compared to existing methods—including PSA [7], VerifyNet [26], EDA [48], FSA [46], VOSA [27], VPPFL [30], and MSA [35], our scheme provides a more comprehensive set of security features.

A. Verifiable Aggregation Via Linear Homomorphic Hash

The verifiability of aggregation results is a critical component for ensuring privacy in federated learning [26]. If an adversary could manipulate the aggregated model returned to users, it could lead to serious privacy breaches [49]. For instance, through carefully crafted malicious updates, the server may deceive users into revealing more sensitive local data [28].

In addition to privacy concerns, a dishonest aggregation server might intentionally perform incorrect or approximate computations to reduce its computational burden, thereby compromising the integrity of the global model [27].

To mitigate these threats, schemes such as VerifyNet [26], VOSA [27], VPPFL [30], and our proposed approach incorporate mechanisms for verifying the correctness of aggregation results. These methods allow users to independently check whether the received model update has been correctly computed, strengthening trust and protecting user privacy. In contrast, existing solutions like PSA [7], EDA [48], FSA [46], and MSA [35] do not support such verification, leaving them more vulnerable to manipulation by untrusted servers.

B. Efficient FL by Clipping-Aware Packed Encoding

In federated learning, significant communication overhead is introduced by the need to transmit updated local models during every training round [50]. Users need to transmit their updated models to the server in every iteration, and modern neural networks, particularly large language models, can be extremely large, sometimes reaching gigabyte-scale sizes [51]. To address this bottleneck, our scheme integrates two key techniques: gradient clipping and packed parameter encoding.

First, we employ gradient clipping to limit the influence of unusually large gradients. Beyond compressing communication data, this approach also enhances defense mechanisms against gradient inversion attacks, which involve reconstructing user private data from gradient information. Compared to existing methods such as EDA [48], FSA [46], VOSA [27], VPPFL [30], and MSA [35], our clipping-based approach achieves a substantial reduction in communication overhead while simultaneously improving privacy protection.

Following clipping, our scheme further compresses model updates using packed parameter encoding. Rather than encrypting each data of local model individually, this technique combines multiple pieces of local model into a single ciphertext, significantly reducing the total number of ciphertexts required for transmission. In contrast, schemes such as EDA [48], FSA [46], VOSA [27], and VPPFL [30] convert gradients into integers before encryption, leading to larger ciphertext sizes due to the inherent expansion in homomorphic encryption. While PSA [7] and VerifyNet [26] transmit floating-point model parameters,

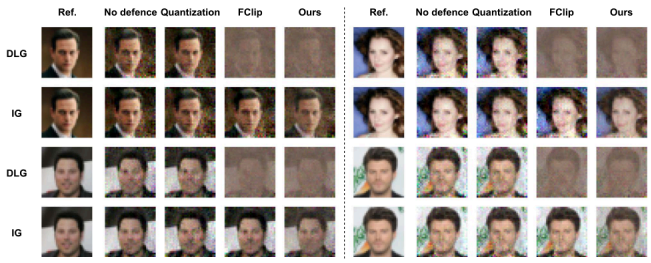
Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
QUANTITATIVE COMPARISON OF STATE-OF-THE-ART GRADIENT INVERSION ATTACKS UNDER VARIOUS DEFENSES

Model+Dataset	Methods	No Defence			Quantization			FClip			Ours		
		PSNR ↑	MSE ↓	LPIPS↓	PSNR ↑	MSE ↓	LPIPS↓	PSNR ↑	MSE ↓	LPIPS↓	PSNR ↑	MSE ↓	LPIPS↓
LeNet+MNIST	DLG [3]	41.350	0.0001	0.0001	40.792	0.0002	0.0001	10.486	0.1061	0.4267	10.980	0.0876	0.3964
	IG [52]	70.210	2.293E-07	2.837E-07	56.616	2.68E-06	3.594E-06	70.109	2.318E-07	2.738E-07	30.288	0.0010	0.0026
LeNet+CIFAR-10	DLG [3]	15.863	0.0355	0.0492	15.765	0.0377	0.0511	8.273	0.1553	0.2153	8.290	0.1540	0.2133
	IG [52]	21.308	0.0145	0.0162	21.244	0.0146	0.0163	21.255	0.0146	0.0162	18.238	0.0188	0.0203
ResNet18+CelebA	DLG [3]	22.175	0.0073	0.0108	22.652	0.0067	0.0125	13.370	0.0512	0.2347	13.343	0.0516	0.2348
	IG [52]	24.637	0.0041	0.0052	23.719	0.0052	0.0082	24.529	0.0045	0.0055	20.820	0.0095	0.0250



(a) LeNet+CIFAR-10



(b) ResNet18+CelebA

Fig. 2. Reconstruction outcomes of two gradient inversion attacks (DLG and IG) under four defense settings: no defense, quantization, clipping (Fclip), and our combined defense. Each block displays the target image followed by reconstructions under different defenses on the LeNet+CIFAR-10 and ResNet18+CelebA configurations. The results show that both quantization and clipping partially degrade inversion quality, whereas our combined defense consistently yields the most significant reduction in reconstruction fidelity across both attacks and models.

they do not leverage packing strategies for communication efficiency.

Although MSA [35] also employs packing technique, our approach uniquely integrates gradient clipping with packed encoding in a unified framework. This combination offers superior communication efficiency while preserving model accuracy and enhancing privacy protection, which is suitable for large-scale and privacy-critical federated learning scenarios.

C. Gradient Inversion Attack Resistance

As shown in Fig. 2 and Table IV, the proposed clipping-aware packed parameter encoding method demonstrates strong resistance to gradient inversion attacks, particularly against Deep Leakage from Gradients (DLG) [3], while providing moderate protection against Inverting Gradients (IG) [52]. We evaluate the reconstruction quality using MSE (↓), PSNR (↑), and LPIPS (↓), which jointly assess pixel-level and perceptual similarity.

Effect of quantization: Quantization alone offers limited protection against DLG and only moderate defense against IG in simple settings (e.g., LeNet+MNIST), while becoming ineffective on more complex datasets such as CIFAR-10. This is because quantization primarily reduces numerical precision and introduces relatively weak perturbations, which are insufficient to disrupt iterative gradient matching in DLG or direction-preserving optimization in IG.

Effect of clipping: Clipping significantly degrades the effectiveness of DLG by constraining gradient magnitudes and discarding critical amplitude information required for accurate reconstruction. However, it has limited impact on IG, which mainly relies on gradient direction rather than magnitude, and thus remains largely unaffected by clipping alone.

Combined defense: By jointly integrating quantization and clipping, the proposed encoding method achieves stronger resistance to DLG and improved robustness against IG compared to either technique alone. This combined defense is particularly effective on structurally simple datasets such as MNIST, where the interaction between magnitude suppression and quantization noise more severely disrupts gradient inversion. Overall, these results validate the effectiveness of the proposed clipping-aware packed parameter encoding in enhancing privacy and security against gradient inversion attacks.

D. Communication Rounds

Following the convention in secure aggregation literature, we count only latency-critical, sequential client–server interactions within each FL iteration as communication rounds, excluding offline setup and non-dependent broadcasts (e.g., public key advertisements). Under this definition, our protocol requires 3 communication rounds per iteration, matching the efficiency of EDA [48], FSA [46], VOSA [27], and MSA [35], and improving upon PSA [7], VerifyNet [26], and VPPFL [30], which require 4 rounds.

VI. SECURITY DISCUSSION

We analyze the security of the proposed protocol under both honest-but-curious and actively malicious adversarial models. Our proofs follow the standard simulation-based paradigm: for each adversarial setting, we show that the adversary's view can be efficiently simulated using only the information that is explicitly allowed to be leaked. All detailed hybrid arguments and simulator constructions are deferred to the appendix, available online.

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

A. Privacy Under Honest-but-Curious Adversaries

We first consider a setting where all parties, including the server and the clients, follow the protocol specification, but may attempt to infer additional information from the protocol transcript.

Theorem 3 (Security against honest-but-curious users and server): For any security parameter κ, any user set $\mathcal{U}$, and any coalition $\mathcal{C} \subseteq \mathcal{U}$ with $|\mathcal{C}| \leq n-2$, and for all local models $\boldsymbol{\omega}_{\mathcal{U}}$ of users, and $\boldsymbol{\omega}_{\mathcal{C}}$ are the local models of the corrupted clients, there exists a PPT simulator SIM such that

$$\mathsf{SIM}_{\mathcal{C}}^{\mathcal{U},\kappa}(\boldsymbol{\omega}_{\mathcal{C}}, \sum_{u \in \mathcal{U}\setminus\mathcal{C}} \boldsymbol{\omega}_u) \approx_c \mathsf{REAL}_{\mathcal{C}}^{\mathcal{U},\kappa}(\boldsymbol{\omega}_{\mathcal{U}}, \boldsymbol{\omega}_{\mathcal{U}}).$$

The view of any honest-but-curious coalition can be simulated using only the inputs of the corrupted users. Consequently, the coalition learns no information about the individual inputs of honest users beyond what is implied by protocol correctness.

B. Privacy Against Actively Malicious Users and Server

We next consider a stronger adversarial model, where the adversary may arbitrarily deviate from the protocol, corrupt the server, and corrupt up to $n-2$ users. The adversary is fully adaptive and controls message scheduling, but cannot break the underlying cryptographic primitives.

Theorem 4 (Privacy against actively malicious users and server): For every PPT adversary $\mathcal{M}_{\mathcal{C}}$ corrupting a set $\mathcal{C} \subseteq \mathcal{U} \cup \{\text{server}\}$ with $|\mathcal{C} \cap \mathcal{U}| \leq n-2$, and for all local models $\boldsymbol{\omega}_{\mathcal{U}\setminus\mathcal{C}}$ of honest users, and $\boldsymbol{\omega}_{\mathcal{C}}$ are the local models of the corrupted clients, there exists a PPT simulator SIM such that

$$\mathsf{SIM}_{\mathcal{C}}^{\mathcal{U},\kappa}\left(\mathcal{M}_{\mathcal{C}}, \boldsymbol{\omega}_{\mathcal{C}}, \sum_{u \in \mathcal{U}\setminus\mathcal{C}} \boldsymbol{\omega}_u\right) \approx_c \mathsf{REAL}_{\mathcal{C}}^{\mathcal{U},\kappa}\left(\mathcal{M}_{\mathcal{C}}, \boldsymbol{\omega}_{\mathcal{U}\setminus\mathcal{C}}\right),$$

i.e., the adversary learns nothing beyond its own inputs and the aggregate sum of the honest users' inputs.

Even a malicious adversary controlling the server and a subset of users learns nothing beyond its own inputs and the aggregate sum of honest users' inputs. In particular, individual honest updates remain computationally hidden.

C. Aggregation Integrity

Integrity is ensured by digital signatures and a linear homomorphic hash (LHH). Each client signs the hash of its local update, preventing the server from forging or altering authenticated hashes, while LHH guarantees that the aggregation of hashes matches the hash of the aggregated update. Thus, any modification or reordering of authenticated updates is detected.

The protocol guarantees that any accepted aggregation equals the sum of authenticated client updates. In the cross-silo setting, rounds are completed with a fixed set of participants; otherwise, the round is aborted and restarted. The mechanism ensures correctness of aggregation but does not prevent malicious clients from submitting incorrect updates. A formal proof is provided in Appendix B, available online.

TABLE V
SUMMARY OF KEY EXPERIMENTAL PARAMETERS

Parameter	IID		Non-IID ($\alpha = 0.5$)	
	MNIST	CIFAR-10	MNIST	CIFAR-10
Learning rate (η)	0.001	0.001	0.01	0.005
Optimizer	Adam	Adam	SGD	SGD
Batch size	64	64	64	64
Local epochs (E)	2	2	5	5
Global epochs	100	100	100	100
No. of clients (n)	10	10	100	100
Fraction of clients	0.8	0.8	0.1	0.1

Theorem 5 (Aggregation correctness and verifiability): Assume that (i) the digital signature scheme is existentially unforgeable under chosen-message attacks (EUF-CMA), and (ii) the linear homomorphic hash (LHH) scheme is binding under the q-Strong Diffie–Hellman assumption. Then, except with negligible probability, any aggregation result $\boldsymbol{\omega}$ accepted by honest users equals the sum of honestly generated local models, namely, $\boldsymbol{\omega} = \sum_{u=1}^{n} \boldsymbol{\omega}_u$.

VII. EVALUATION

The implementation of our scheme is carried out in Java and deployed on a Windows 10 workstation equipped with an $Intel^{(R)}\ Core^{(TM)}$ i9-10920X (3.50 GHz) processor and 32 GB of RAM. The bit length of N, which is used in decryption, is 1024 bits. We use the Java Pairing-Based Cryptography Library (JPBC) to realize the linear homomorphic hash. The order of the elliptic curve is 256 bits, aligning with the NIST recommendations for elliptic curve key lengths for the period 2019-2030 [53].

Dataset: We explore attack models trained on various datasets, encompassing MNIST, CIFAR-10, and CelebA.

Model: To assess the effect of quantization on accuracy, we conducted federated learning experiments using both CNN and LeNet models. In our scheme, the CNN architecture comprises four convolutional layers and one fully connected (FC) layer, and the LeNet architecture includes two convolutional layers, two pooling layers, and three FC layers.

Federated Learning Setting: The main experimental configurations are summarized in Table V. The batch size is 64 and the global training epoch is 100, while gradients are quantized to 32 bits with scaling factor $\lambda = 10^4$. For IID settings (MNIST and CIFAR-10), Adam is used with a learning rate of 10^{-3}, 2 local epochs, 10 clients, and a participation ratio of 0.8. For Non-IID settings ($\alpha = 0.5$), SGD is applied with learning rates of 10^{-2} (MNIST) and 5×10^{-3} (CIFAR-10), 5 local epochs, 100 clients, and a participation ratio of 0.1.

A. Comparison of Computation Cost

We analyze the computation overhead of the proposed scheme from a structural and quantitative perspective, and compare it with PSA [7], MSA [35], VerifyNet [26], and VOSA [18]. As illustrated in Fig. 3, the proposed scheme does not minimize per-round computation in all settings, but achieves a carefully

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
COMPARISON OF COMPUTATION COMPLEXITY WITH INCREASING NUMBER OF USERS. THE MSA IS INCLUDED AS A NON-VERIFIABLE LOWER-BOUND BASELINE FOR COMPUTATIONAL COST

Protocols		PSA [7]			VerifyNet [26]			VOSA [27]			MSA [35]			Ours		
Client Number		25	75	125	25	75	125	25	75	125	25	75	125	25	75	125
Advertise keys	Client	14.06	98.65	216.43	6.24	54.32	126.61	0	0	0	0.01	0.02	0.04	0.14	0.45	0.67
Masked Inputs	Client	5.79	41.77	93.76	2050.04	5846.87	8321.09	1460.69	3723.71	6017.27	3.15	9.40	15.14	36.25	109.72	166.75
Unmasking	Server	0.038	0.53	1.42	2.95	9.82	14.44	40.93	37.50	40.77	0.013	0.043	0.071	0.046	0.14	0.21
	Client	13.80	98.10	220.98	6.21	54.20	127.82	0	0	0	0.074	0.21	0.39	0.13	0.40	0.61
Verification	Client	-	-	-	2714.80	7787.59	10830.11	981.47	2540.69	4197.82	-	-	-	33.79	129.67	233.50
Total time		33.96	239.72	538.00	4780.29	13752.93	19420.27	2484.10	6304.51	10260.20	3.30	9.82	15.86	70.41	240.53	402.00

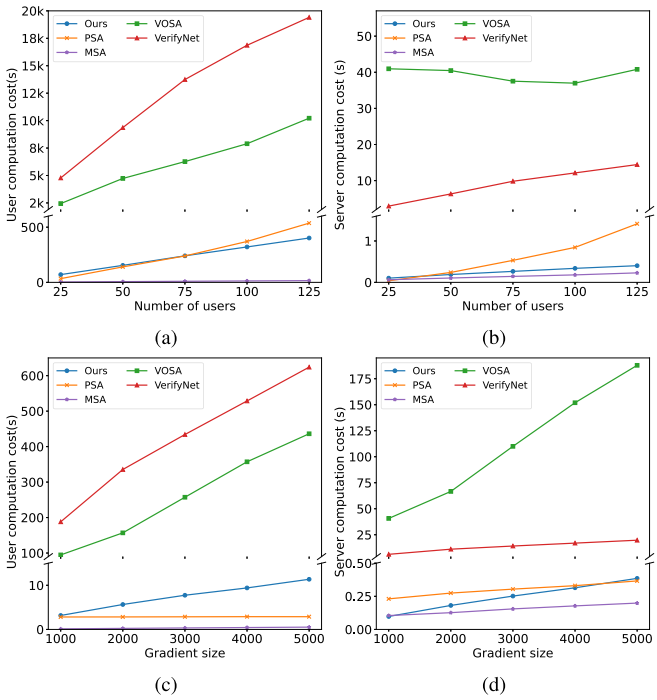


Fig. 3. Computation cost under varying system scales. Figs. (a) and (b) evaluate user-side and server-side computation costs as the number of users N increases (fixed gradient dimension 1000). Figs. (c) and (d) illustrate the impact of increasing gradient dimension (fixed $n = 50$). The proposed scheme consistently outperforms VOSA and VerifyNet by orders of magnitude. This reduction is attributed to the elimination of expensive per-gradient bilinear pairing operations used in VerifyNet and VOSA, and the efficiency of the proposed *clipping-aware packed parameter encoding*. Compared to PSA and MSA, our scheme maintains competitive efficiency while supporting enhanced verification capabilities.

balanced trade-off between efficiency, scalability, and verifiable security, which directly reflects our design goals.

To enable a systematic comparison, we decompose all schemes into four phases: 1) Setup & Advertise Keys, 2) Masked Inputs, 3) Unmasking, and 4) Verification. We analyze the computation cost with respect to both the number of users and the gradient dimensionality.

Impact of the Number of Users: Fig. 3(a) and Fig. 3(b)show the computation cost as the number of participating clients increases. In the *Setup & Advertise Keys* phase, our scheme incurs consistently low overhead across all user scales. Notably, this phase is executed in every training round to guarantee key freshness and prevent cross-round linkability, rather than being a one-time initialization cost. PSA and VerifyNet, which rely on double-masking and secret-share encryption, exhibit significantly higher overhead that grows with the number of users.

In the *Masked Inputs* phase, VerifyNet and VOSA incur substantially higher computation cost due to per-gradient verification and pairing-based operations. Our scheme remains significantly more efficient in this phase. Compared with PSA, our scheme incurs higher cost when the number of users is small (e.g., fewer than 75 clients), mainly due to LHH construction. However, PSA's computation grows rapidly with the number of users because of the increasing number of double-maksing key sharing, whereas the cost of our scheme scales approximately linearly. As a result, our scheme becomes more efficient than PSA in larger-scale settings. MSA exhibits the lowest computation cost in this phase, as it only encrypts gradients and does not support aggregation verification.

In the *Unmasking* and *Verification* phases, users in our scheme only generate and transmit decryption keys, while the server aggregates encrypted gradients and verification information. In contrast, PSA and VerifyNet require reconstructing secret shares and performing decryption, leading to rapidly increasing overhead as the number of users grows. VerifyNet and VOSA further incur additional pairing-based verification, making them the least efficient schemes in this phase. Although our scheme introduces extra computation compared with MSA, this overhead remains moderate and scales with the number of users rather than the gradient size.

Impact of Gradient Size: Fig. 3(c) and Fig. 3(d) illustrate the effect of increasing gradient size. VerifyNet and VOSA perform verification for each gradient element, causing their computation cost to increase sharply as gradient dimensionality grows. Although the LHH computation in our scheme also scales with gradient size, the combined use of clipping-aware packed parameter encoding and Accuracy-Adaptive Model Transmission (AAMT) significantly mitigates this growth.

On the user side, the computation cost of our scheme remains lower than PSA for moderate gradient sizes and only exceeds PSA when the gradient dimension exceeds approximately 2000. On the server side, our scheme is only slower than PSA when the gradient size reaches around 5000. This is because homomorphic aggregation cost increases with ciphertext size, whereas PSA

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

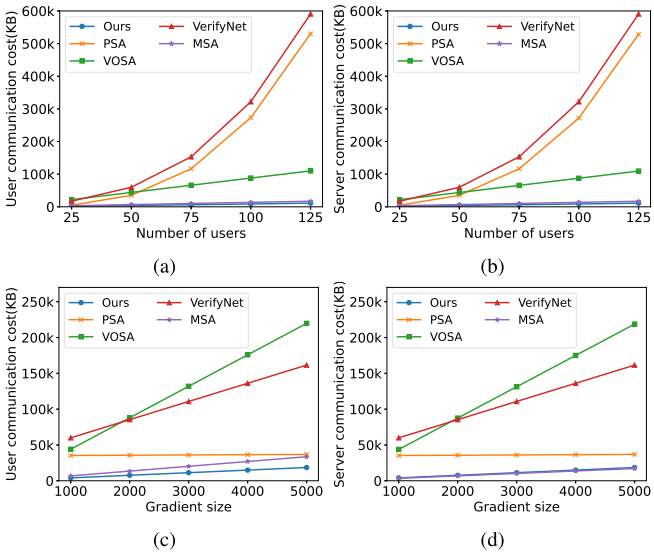


Fig. 4. Comparative analysis of communication overhead under varying system scales. Figs. (a) and (b) depict the user-side and server-side communication costs, respectively, as the number of users increases (fixed gradient size 1000). Figs. (c) and (d) illustrate the costs as the gradient size varies (fixed user count $n = 50$). The proposed scheme ("Ours") demonstrates superior scalability compared to VerifyNet, PSA, and VOSA, and remains competitive with MSA. This efficiency is primarily attributed to the proposed *clipping-aware packed parameter encoding* and *Accuracy-Adaptive Model Transmission (AAMT)*, which effectively mitigate the ciphertext expansion inherent in homomorphic encryption.

reconstructs secret shares with computation cost largely independent of gradient dimensionality but higher initial overhead. Compared with MSA, our scheme incurs additional computation due to verification; however, this overhead enables public verifiability and robustness against malicious aggregation, which are not provided by MSA.

Summary: The computation cost of the proposed scheme has been evaluated across all major FL phases under varying numbers of users and gradient sizes. While PSA and MSA achieve lower per-round costs without verifiable aggregation, their overhead either scales poorly with user population or lacks robustness against adversarial behavior. Our scheme introduces moderate and bounded additional cost to support verifiability and attack resistance, which remains stable across user and gradient scales, and is partially offset by avoiding secret-share reconstruction and leveraging accuracy-adaptive model transmission. Consequently, MSA serves as a non-verifiable lower-bound baseline, whereas our approach balances computational efficiency, scalability, and strong security across the full FL lifecycle.

B. Comparison of Communication Cost

We analyze the communication efficiency of the proposed scheme from a structural perspective and compare it with representative secure aggregation methods. As illustrated in Fig. 4, our scheme consistently achieves lower communication overhead, despite incorporating homomorphic encryption and verifiable aggregation. This advantage arises from a careful co-design of aggregation, verification, and transmission mechanisms, where both gradient representation and transmission frequency are jointly optimized. In particular, communication efficiency is achieved not only through compact ciphertext construction, but also through adaptive reduction of transmitted model components during training.

Impact of Gradient Size: In homomorphic-encryption-based schemes, communication cost typically grows rapidly with gradient size due to the transmission of large ciphertexts. To mitigate this bottleneck, we propose a *clipping-aware packed parameter encoding* method (Contribution 3), which embeds multiple clipped gradients into a single ciphertext. As a result, the number of transmitted ciphertexts increases sub-linearly with gradient dimensionality. In contrast, VOSA applies homomorphic encryption without gradient packing, leading to the highest communication overhead as gradient size grows. Although MSA also adopts an encoding strategy, it incurs additional communication due to the transmission of encoding-related parameters, especially on the server side. Consequently, our scheme maintains lower communication cost under large gradient dimensions, as shown in Fig. 4(c) and Fig. 3(d).

Impact of the Number of Users: We further examine communication cost as the number of participating clients increases. Schemes such as PSA and VerifyNet rely on Shamir secret sharing to reconstruct keys or masked gradients, requiring the transmission of secret shares proportional to the number of users. This design causes their communication overhead to scale rapidly with client population. In contrast, our scheme adopts a practical multi-private-key aggregation architecture (Contribution 1), where increasing the number of users mainly affects the number of ciphertexts, signatures, and verification messages received by the server. Despite larger ciphertext sizes, the overall communication growth remains moderate, resulting in superior scalability, as illustrated in Fig. 4(a) and Fig. 3(b).

Accuracy-Adaptive Transmission with Verifiable Aggregation: Beyond static communication reduction, the proposed scheme further improves communication efficiency through Accuracy-Adaptive Model Transmission (AAMT), which dynamically adjusts transmitted model components according to convergence behavior. Unlike PSA, which does not support result verification, our scheme integrates linear homomorphic hash (LHH) verification (Contribution 2) while maintaining moderate communication overhead. Owing to the linearity of the hash construction, the additional verification messages are lightweight and remain significantly less expensive than per-gradient verification schemes such as VerifyNet and VOSA.

More importantly, AAMT enables selective transmission of only fully connected (FC) layers once global accuracy stabilizes, effectively reducing communication volume in later training stages. As reported in Table VII , AAMT is activated in a substantial fraction of training rounds, with empirical activation rates ranging from approximately 30% to over 55% under IID settings, depending on the model and dataset. Under non-IID data, activation rates decrease due to more fluctuating accuracy trajectories caused by data heterogeneity, yet AAMT remains frequently exercised rather than an occasional optimization. This adaptive behavior translates into tangible bandwidth savings:

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
AAMT ACTIVATION RATES AND BANDWIDTH SAVINGS

Configuration	Activation Rate		Bandwidth Saving	
	IID	Non-IID	Ratio	Params
CNN+MNIST	41.38%	44.2%	65.43%	3.69M
LeNet+MNIST	41.25%	30.5%	15.44%	4.37M
CNN+CIFAR-10	55.88%	9.9%	60.44%	5.25M
LeNet+CIFAR-10	32.5%	9.8%	11.59%	3.65M

TABLE VIII
AAMT TRIGGERING THRESHOLDS. TRIGGERS DEPEND ON ACCURACY VARIATION (δ)

Setting	Dataset	Model	Trigger Condition
IID	MNIST	CNN	$\lvert\delta\rvert < 0.45\%$
	CIFAR-10	CNN	$\lvert\delta\rvert < 1.3\%$
	MNIST	LeNet	$\lvert\delta\rvert < 0.45\%$
	CIFAR-10	LeNet	$\lvert\delta\rvert < 2.5\%$
Non-IID	MNIST	CNN	$\lvert\delta\rvert < 2.5\%$
	CIFAR-10	CNN	$\lvert\delta\rvert < 1.3\%$
	MNIST	LeNet	$\lvert\delta\rvert < 2.5\%$
	CIFAR-10	LeNet	$\lvert\delta\rvert < 3\%$

for CNN-based models, skipping convolutional layers during AAMT-activated rounds saves approximately 3.7 M parameters on MNIST and over 5.2 M parameters on CIFAR-10, whereas LeNet achieves smaller absolute savings (around 3.6–4.4 M parameters) since a larger portion of its parameters reside in FC layers that are still transmitted. Overall, these results demonstrate that AAMT provides substantial and architecture-dependent bandwidth reduction while remaining fully compatible with verifiable aggregation.

Summary: By jointly optimizing gradient encoding, aggregation scalability, verification efficiency, and transmission adaptivity, the proposed scheme achieves a favorable communication–security trade-off. In contrast to schemes that rely solely on static compression or omit verifiability, our design reduces communication overhead both structurally and temporally, while preserving aggregation correctness and robustness against gradient-based attacks (Contribution 4).

C. Comparison of Model Accuracy

To evaluate the influence of Accuracy-Adaptive Model Transmission (AAMT), clipping-aware packed parameter encoding (we mainly tested FClip and LClip) on model accuracy, we utilize CNN and LeNet architectures for training on the MNIST and CIFAR-10 datasets. The results, presented in Fig. 5, demonstrate that these methods introduce only negligible accuracy degradation, confirming their practical viability.

On MNIST, all methods achieve nearly identical accuracy under the IID setting, demonstrating that AAMT and clipping have negligible impact on convergence for simple datasets. Under the non-IID setting, clipping-based methods lead to moderately lower accuracy for CNN, as clipping suppresses high-norm updates that are important for mitigating distribution bias. This effect is less pronounced for LeNet, whose deeper architecture can better compensate for the information loss introduced by clipping.

On the more complex CIFAR-10 dataset, overall accuracy is lower and performance differences among methods are more visible. In the IID setting, Baseline and Baseline+FClip achieve comparable accuracy, while AAMT-based methods converge to slightly lower performance. For LeNet, Baseline+LClip outperforms most alternatives except the Baseline, indicating that the impact of clipping depends on network depth and structure. Under the non-IID setting, all methods experience a notable accuracy drop due to increased statistical heterogeneity, while the relative ranking of methods remains consistent with MNIST.

Due to the higher complexity of CIFAR-10, the accuracy gap between the best- and worst-performing methods under non-IID data reaches nearly 10%, compared with approximately 6% under IID. This gap is primarily caused by data heterogeneity rather than by AAMT or clipping. Overall, the additional accuracy loss introduced by the proposed clipping-aware packed parameter encoding remains limited and acceptable.

Summary: These results confirm that the proposed encoding method enhances communication efficiency and resistance to gradient inversion and data poisoning attacks, while maintaining competitive accuracy even on complex datasets and under non-IID distributions.

D. Comparison of Accuracy Under Poison Attack

The experimental results in Fig. 6 evaluate model accuracy under non-backdoor settings and attack success rate (ASR) under backdoor poisoning attacks defined in [38]. We conduct experiments using CNN and LeNet models on MNIST and CIFAR-10, and compare our methods with representative robust aggregation defenses, including Krum [54], Median, and Trimmed_Mean [55].

The results reveal clear trade-offs among existing defenses. Krum and Median largely preserve model accuracy but offer limited resistance to backdoor attacks, as reflected by relatively high ASR, particularly on CIFAR-10. In contrast, Trimmed_Mean significantly suppresses the backdoor attack, achieving lower ASR at the cost of substantial accuracy degradation on complex tasks. These observations highlight the robustness–utility trade-off inherent in aggregation-based defenses.

Among clipping strategies, flat clipping (FClip) consistently outperforms layer-wise clipping (LClip) by achieving lower ASR while maintaining comparable accuracy. By enforcing a global constraint across layers, FClip more effectively disrupts malicious gradient perturbations, whereas LClip allows certain backdoor signals to survive due to its layer-local normalization. Accuracy-Adaptive Model Transmission (AAMT) further improves robustness by filtering low-quality updates and is most effective when combined with clipping, while AAMT alone provides limited protection.

Quantization, implemented via clipping-aware packed parameter encoding, introduces additional noise that modestly enhances backdoor resistance while improving communication efficiency. Overall, no single defense achieves both optimal accuracy and robustness. Compared with aggregation-based

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

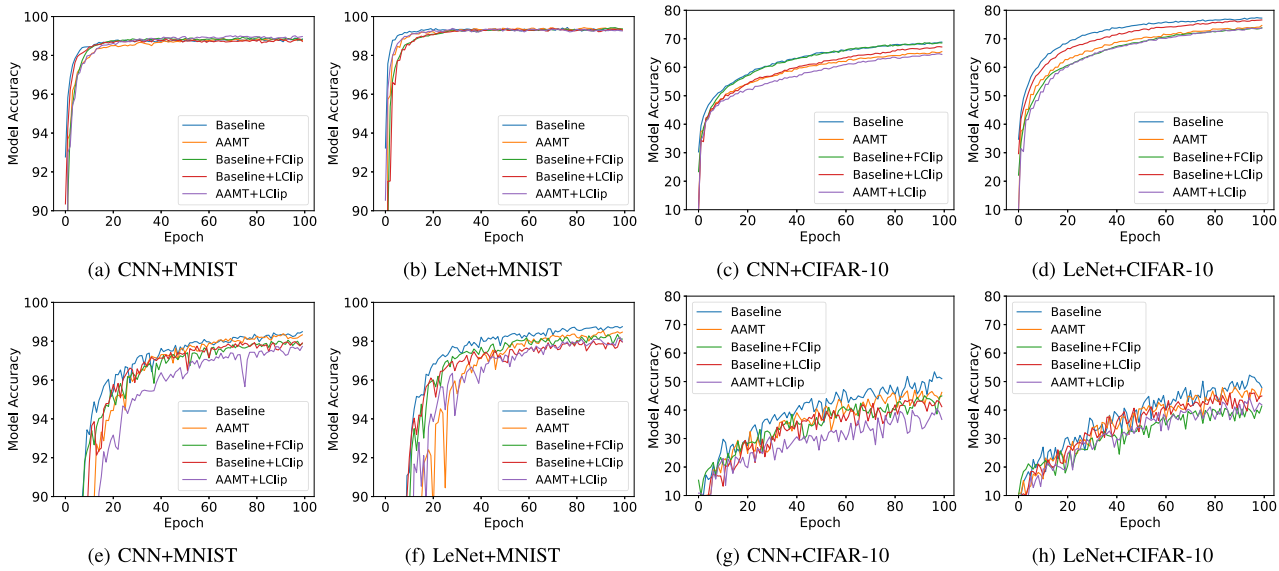


Fig. 5. Effect of data heterogeneity on model accuracy. Subfigures (a) and (b) report the accuracy of CNN and LeNet on MNIST under the IID setting, while (e) and (f) present the corresponding results under the non-IID setting. Similarly, (c) and (d) show the accuracy on CIFAR-10 under IID data distributions, and (g) and (h) correspond to the non-IID case. Across both datasets, non-IID data leads to noticeable accuracy degradation, especially on CIFAR-10, while the relative performance ordering of different methods remains consistent. Both flat clipping (FClip) and layer-wise clipping (LClip) include clipping-aware quantization, and the additional accuracy loss introduced by clipping and accuracy-adaptive transmission is moderate compared with the impact of data heterogeneity.

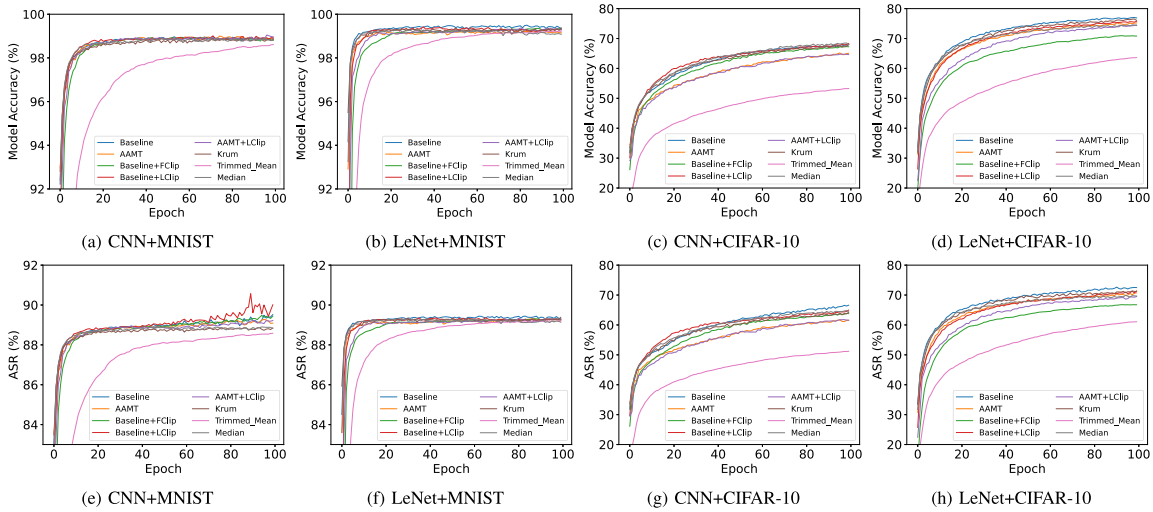


Fig. 6. Model behavior under backdoor poisoning attacks. Subfigures (a) and (b) report the classification accuracy on benign test samples without triggers, reflecting the preservation of normal semantic functionality under poisoning. Subfigures (e) and (f) report the classification accuracy on backdoored test samples with triggers, corresponding to backdoor effectiveness. Subfigures (c) and (d) and (g) and (h) present the same evaluation on CIFAR-10. Results compare the proposed clipping-based and accuracy-adaptive defenses with robust aggregation baselines. Both flat clipping (FClip) and layer-wise clipping (LClip) include clipping-aware quantization, illustrating the trade-off between semantic accuracy preservation and backdoor suppression.

methods such as Trimmed_Mean, the combination of FClip, AAMT, and quantization offers a more balanced trade-off, particularly on CIFAR-10, while simpler datasets like MNIST benefit primarily from clipping-based defenses.

VIII. Conclusion

In this study, we tackled the critical challenge of balancing privacy and efficiency in federated learning with large-scale deep learning models. To this end, we proposed a novel multi-key secure aggregation scheme which enables efficient and privacy-preserving model aggregation. Furthermore, we developed a novel parameter encoding method that significantly compresses high-dimensional model updates. This compression not only dramatically decreases the communication bandwidth but also improves the resistance to gradient inversion attacks and backdoor attacks in federated learning. Experimental evaluations validate that our scheme outperforms existing approaches in terms

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

of both security and practical deployment feasibility. These findings suggest that the proposed method is well-suited for deploying large-scale deep learning models in privacy-sensitive environments.

REFERENCES

[1] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Denver, Colorado, USA:ACM Press, 2015, pp. 1310–1321.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, Apr. 2017, pp. 1273–1282.

[3] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. NeurIPS*, 2019, pp. 14774–14784.

[4] J. Jeon, j. Kim, K. Lee, S. Oh and J. Ok, "Gradient inversion with generative image prior," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 29898–29908.

[5] H. Fang, B. Chen, X. Wang, Z. Wang, and S.-T. Xia, "GIFD: A generative gradient inversion method with feature domain optimization," in *Proc. 2023 IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4944–4953.

[6] G. Zhang, B. Liu, H. Tian, T. Zhu, M. Ding, and W. Zhou, "How does a deep learning model architecture impact its privacy? A comprehensive study of privacy attacks on CNNs and transformers," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA: USENIX Association, Aug. 2024, pp. 6795–6812. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-guangsheng

[7] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA:Association for Computing Machinery, 2017, pp. 1175–1191, doi: 10.1145/3133956.3133982.

[8] J. Song, W. Wang, T. R. Gadekallu, J. Cao, and Y. Liu, "EPPDA: An efficient privacy-preserving data aggregation federated learning scheme," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 3047–3057, Sep./Oct. 2023.

[9] T. Stevens, C. Skalka, C. Vincent, J. Ring, S. Clark, and J. Near, "Efficient differentially private secure aggregation for federated learning via hardness of learning with errors," in *Proc. 31st USENIX Secur. Symp.*. Boston, MA, USA:USENIX, 2022, pp. 1379–1395.

[10] Y. Ma, J. Woods, S. Angel, A. Polychroniadou, and T. Rabin, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proc. IEEE Symp. Secur. Privacy*, May 2023, pp. 477–496.

[11] J. Bell et al., "ACORN: Input validation for secure aggregation," in *Proc. 32nd USENIX Secur. Symp.*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 4805–4822.

[12] K. N. Kumar, R. Mitra, and C. K. Mohan, "Revamping federated learning security from a defender's perspective: A unified defense with homomorphic encrypted data space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2024, pp. 24387–24397.

[13] C. Hu and B. Li, "MaskCrypt: Federated learning with selective homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 221–233, Jan./Feb. 2025.

[14] M. Crawshaw and M. Liu, "Federated learning under periodic client participation and heterogeneous data: A new communication-efficient algorithm and analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 8240–8299.

[15] Y. Deng et al., "A communication-efficient hierarchical federated learning framework via shaping data distribution at edge," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2600–2615, 2024.

[16] F. Wu, Z. Li, Y. Li, B. Ding, and J. Gao, "FedBiOT: LLM local fine-tuning in federated learning without full model," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA:Association for Computing Machinery, 2024, pp. 3345–3355, doi: 10.1145/3637528.3671897.

[17] W. Kuang et al., "FederatedScope-LLM: A comprehensive package for fine-tuning large language models in federated learning," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*. New York, NY, USA:Association for Computing Machinery, 2024, pp. 5260–5271, doi: 10.1145/3637528.3671573.

[18] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.

[19] H. Gao, N. He, and T. Gao, "SVeriFL: Successive verifiable federated learning with privacy-preserving," *Inf. Sci.*, vol. 622, pp. 98–114, Apr. 2023.

[20] M. Zhang, S. Chen, J. Shen, and W. Susilo, "PrivacyEAFL: Privacy-enhanced aggregation for federated learning in mobile crowdsensing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5804–5816, 2023.

[21] J. Tang, H. Xu, M. Wang, T. Tang, C. Peng, and H. Liao, "A flexible and scalable malicious secure aggregation protocol for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4174–4187, 2024.

[22] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *Proc. 14th USENIX Symp. Netw. Syst. Des. Implementation*, Boston, MA: USENIX Association, Mar. 2017, pp. 259–282. [Online]. Available: https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/corrigan-gibbs

[23] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Prio: Privacy preserving aggregate statistics via Boolean shares," in *Security and Cryptography for Networks*, C. Galdi and S. Jarecki, Eds. Cham, Switzerland: Springer, 2022, pp. 516–539.

[24] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," in *Proc. IEEE Symp. Secur. Privacy*, May 2023, pp. 1961–1979.

[25] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=SkhQHMW0W

[26] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.

[27] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.

[28] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*. New York, NY, USA:Association for Computing Machinery, Oct. 2017, pp. 603–618.

[29] L. H. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," in *Proc. Int. Conf. Learn. Representations*, 2022.

[30] X. Shen et al., "Verifiable privacy-preserving federated learning under multiple encrypted keys," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3430–3445, Jan. 2024.

[31] J. Cai, W. Shen, and J. Qin, "ESVFL: Efficient and secure verifiable federated learning with privacy-preserving," *Inf. Fusion*, vol. 109, 2024, Art. no. 102420.

[32] S. Li, X. Wei, and H. Wang, "OVP-FL: Outsourced verifiable privacy-preserving federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 3, pp. 2057–2069, May/Jun. 2025.

[33] J. Wu, W. Zhang, and F. Luo, "ESAFL: Efficient secure additively homomorphic encryption for cross-silo federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3868–3884, Jul./Aug. 2025.

[34] R. Wang, L. Xiong, J. Geng, C. Xie, and R. Li, "An effective and verifiable secure aggregation scheme with privacy-preserving for federated learning," *J. Syst. Archit.*, vol. 161, 2025, Art. no. 103364.

[35] X. Yang, Z. Liu, X. Tang, R. Lu, and B. Liu, "An efficient and multi-private key secure aggregation scheme for federated learning," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 1998–2011, Sep./Oct. 2024.

[36] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*. New York, NY, USA:Association for Computing Machinery, 2017, pp. 603–618, doi: 10.1145/3133956.3134012.

[37] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *Proc. Adv. Cryptology - ASIACRYPT 2003*, C.-S. Laih Ed. Berlin, Germany: Springer, 2003, pp. 37–54.

[38] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, "BadNets: Evaluating backdooring attacks on deep neural networks," *IEEE Access*, vol. 7, pp. 47230–47244, 2019.

[39] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Twenty Third Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.

[40] Y. Li, Y. Jiang, Z. Li, and S.-T. Xia, "Backdoor learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 5–22, Jan. 2024.

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.

[41] X. Ma et al., "Repairing backdoor model with dynamic gradient clipping for intelligent vehicles," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 804–818, Jan./Feb. 2025.
[42] J. Bai, K. Gao, S. Min, S.-T. Xia, Z. Li, and W. Liu, "BadCLIP: Trigger-aware prompt learning for backdoor attacks on clip," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2024, pp. 24239–24250.
[43] M. Bellare, O. Goldreich, and S. Goldwasser, "Incremental cryptography: The case of hashing and signing," in *Proc. 14th Annu. Int. Cryptology Conf.*, Santa Barbara, CA, USA, 1994, pp. 216–233.
[44] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," *J. Cryptol.*, vol. 17, no. 4, pp. 297–319, 2004, doi: 10.1007/s00145-004-0314-9.
[45] J. Hu et al., "Does differential privacy really protect federated learning from gradient leakage attacks?," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12635–12649, Dec. 2024.
[46] S. Yang, Y. Chen, Z. Yang, B. Li, and H. Liu, "Fast secure aggregation with high dropout resilience for federated learning," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 3, pp. 1501–1514, Sep. 2023.
[47] J. Liu, Q. Zeng, H. Xu, Y. Xu, Z. Wang, and H. Huang, "Adaptive block-wise regularization and knowledge distillation for enhancing federated learning," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 791–805, Feb. 2024.
[48] Z. Liu, J. Guo, K.-Y. Lam, and J. Zhao, "Efficient dropout-resilient aggregation for privacy-preserving machine learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1839–1854, 2023.
[49] Y. Wen, J. A. Geiping, L. Fowl, M. Goldblum, and T. Goldstein, "Fishing for user data in large-batch federated learning via gradient magnification," in *Proc. 39th Int. Conf. Mach. Learn.*, Jun. 2022, pp. 23668–23684.
[50] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-i.i.d. data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
[51] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. 2016 IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[52] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients - How easy is it to break privacy in federated learning?," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 16937–16947.
[53] E. Barker, "Recommendation for key management: Part 1 - general," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. NIST SP 800-57pt1r5, May 2020.
[54] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus,, S. Vishwanathan, and R. Garnett, Eds. 2017, pp. 118–128.
[55] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.



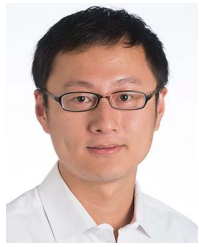
Chenxu Fan received the BE degree from the Wuhan University of Technology, Wuhan, China, in 2023. He is currently working toward the ME degree in computer science and technology with the Beijing University of Technology, Beijing, China. His research interests include the domain of AI security, with an emphasis on federated learning, adversarial learning, and data privacy.



Yuwen Chen (Member, IEEE) received the MS degree in computer software and theory from Zhengzhou University, Zhengzhou, China, in 2015, and the PhD degree in telematic engineering form the Technical University of Madrid, Madrid, Spain, in 2019. He is currently an associate professor with the School of Computer Science, Beijing University of Technology, Beijing, China. His research interests include IoT security and privacy, collaborative learning, model inversion, and gradient inversion.



Qin Jiang received the MS degree in computer science and technology from ZhengZhou University, ZhengZhou, China, in 2016 and the PhD degree from Xi'an Jiaotong University, Xi'an, China, in 2022. She was the PhD intern student with the National University of Singapore. She is currently with the Nanjing University of Information Science and Technology. Her research interests include searchable encryption, trusted hardware, and blockchain.



Haiyang Yu (Member, IEEE) received the PhD degree in computer science and technology from the Beijing University of Technology, in 2019. From 2017 to 2018, he visited the University of Melbourne in Australia. He is currently an associate professor of computer science with the Beijing University of Technology. He has authored or coauthored more than 20 papers in highly ranked journals and top conference proceedings. His research interests include cloud storage auditing, blockchain, and federated learning. He is a member of ACM.



Zhen Yang (Member, IEEE) received the PhD degree in signal processing from the Beijing University of Posts and Telecommunications, Beijing, China. He is currently the full professor of computer science and engineering with the Beijing University of Technology, Beijing. He has authored or coauthored more than 30 papers in highly ranked journals and top conference proceedings. His research interests include data mining, machine learning, trusted computing, and content security. He is the senior member of the Chinese Institute of Electronics.

Authorized licensed use limited to: Anhui University. Downloaded on October 02,2026 at 12:54:25 UTC from IEEE Xplore. Restrictions apply.